

¿Qué son los Hooks de Rendimiento? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Los Hooks de Rendimiento son funciones diseñadas para optimizar la velocidad y la eficiencia de una aplicación. Su objetivo principal es evitar que React haga cálculos costosos o vuelva a dibujar (renderizar) componentes en la pantalla cuando no es necesario.

Los dos principales Hooks de rendimiento en React son useMemo y useCallback.

Características:

- **Memorización (Caching)**: ambos hooks funcionan con "memorización". Guardan en la memoria un resultado o una función y lo reutilizan en los siguientes renderizados, a menos que los datos originales hayan cambiado.
- **Dependencias**: igual que useEffect, reciben un arreglo de dependencias []. React vuelve a calcular el valor o recrear la función solo si alguna de las variables dentro del arreglo de dependencias cambia.

useMemo memoriza el resultado de una operación matemática o un procesamiento de datos costoso (por ejemplo filtrar una lista de 10,000 usuarios).

Por otro lado, useCallback memoriza definiciones de funciones. Es muy útil si se le pasan funciones como props a componentes hijos, ya que evita que el hijo se vuelva a dibujar innecesariamente.

Ejemplo:

En React cada vez que cambia el valor de un estado se vuelve a dibujar la interfaz. En el ejemplo tenemos dos estados independientes, uno que incrementa el número de calculoPesado y otro que solo cuenta los clics. Si nosotros clickeamos en sumar clic, cambiando el estado, y no estuviese definido useMemo sobre la función calculo costoso, la pantalla se redibujaría y se volvería a calcular la función, apesar de ya haber calculado la función con ese mismo número.

En cambio si clickeamos cambiar número de cálculo, si se volvería a ejecutar la función porque en las dependencias del useMemo incluimos la variable numero.

```

import { useState, useMemo } from 'react';

function OptimizacionEjemplo() {
  const [contador, setContador] = useState(0);
  const [numero, setNumero] = useState(5);

  // operación muy lenta y costosa
  const calculoPesado = (num) => {

    let resultado = 0;
    for (let i = 0; i < 10000000000; i++) {
      resultado += num;
    }

    return resultado;
  };

  // useMemo para recordar el resultado y no volver a calcularlo
  // a menos que cambie la variable numero.
  const valorMemorizado = useMemo(() => calculoPesado(numero), [numero]);

  return (
    <div>
      <h2>Hooks de Rendimiento</h2>

      <p>Resultado del cálculo: {valorMemorizado}</p>
      <button onClick={() => setNumero(numero + 1)}>
        Cambiar Número del Cálculo
      </button>

      <hr />

      <p>Clicks en el botón: {contador}</p>
      { /* al hacer clic, el cálculo pesado no se vuelve a ejecutar */ }
      <button onClick={() => setContador(contador + 1)}>
        Sumar Clic
      </button>
    </div>
  );
}

export default OptimizacionEjemplo;

```

¿Qué es el contexto en React? ¿Cuáles son sus características y para qué sirve?
 ¿Qué diferencia existen con los props? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

El contexto (Context API) es una herramienta integrada que permite compartir información globalmente a través de la aplicación sin tener que pasarla manualmente nivel por nivel. Es la solución nativa de React para el prop drilling (pasas props a través de varios niveles de

componentes intermedios que no necesitan esos datos, solo para que lleguen a un componente hijo que si los requiere).

Características:

- **Alcance global**: cualquier componente puede acceder a los datos compartidos, sin importar qué tan profundo esté en el árbol jerárquico.
- **Datos de toda la app**: puede manejar información que necesitan muchos componentes distintos al mismo tiempo (como el usuario que inició sesión, el tema claro/oscuro de la interfaz, o el idioma).
- **Proveedor y consumidor**: hay un componente que funciona como Proveedor que envuelve la aplicación que emite los datos, y un Hook (useContext) que los consume solo donde hacen falta.
Al llamar a la función `createContext()`, React fabrica automáticamente un componente llamado Provider, el programador lo pone en el código envolviendo a los componentes hijos que van a necesitar consumir esa información.

Los props solo pueden pasar datos de componentes padres a hijos inmediatos, mientras que en el contexto está este componente Proveedor que puede pasar información a hijos, sin importar donde esté en el árbol. Por eso los props son para datos más específicos de cada componente y el contexto para datos globales.

Ejemplo:

`createContext()` crea el canal de comunicación de datos para el tema de la interfaz.

`TemaContext.Provider` envuelve la aplicación (el componente App) y usa la propiedad `value` para transmitir el estado actual (tema) y la función para modificarlo (`cambiarTema`).

`useContext(TemaContext)` es el receptor. El componente `BotonMagico` puede leer el contexto y sacar los datos, sin que el componente App haya tenido que usar props para pasárselos.

```

import { createContext, useContext, useState } from 'react';

// hay que crear el Contexto
const TemaContext = createContext();

// componente hijo
function BotonMagico() {
  // aca decimos que el componente esta consumiendo el contexto
  const { tema, cambiarTema } = useContext(TemaContext);

  return (
    <button
      onClick={cambiarTema}
      style={{
        background: tema === 'oscuro' ? '□#333' : '■#FFF',
        color: tema === 'oscuro' ? '■#FFF' : '□#000'
      }}
    >
      Tema actual: {tema}
    </button>
  );
}

// componente padre (el proveedor)
export default function App() {
  const [tema, setTema] = useState('claro');

  const cambiarTema = () => {
    setTema(tema === 'claro' ? 'oscuro' : 'claro');
  };

  return (
    // envolver la app, proveer los datos a través del 'value'
    <TemaContext.Provider value={{ tema, cambiarTema }}>
      <div>
        <h1>Mi Aplicación</h1>
        <BotonMagico />
      </div>
    </TemaContext.Provider>
  );
}

```